

A Service-Oriented Permissioned Blockchain for the Internet of Things

Chao Qiu, *Student Member IEEE*, Haipeng Yao, *Member IEEE*, F. Richard Yu, *Fellow IEEE*, Chunxiao Jiang, *Senior Member IEEE*, and Song Guo, *Senior Member IEEE*

Abstract—Recently, the emergence of blockchain has stirred great interests in the field of Internet of Things (IoT). However, numerous non-trivial problems in the current blockchain system prevent it from being used as a generic platform for large-scale services and applications in IoT. One notable drawback is the scalability problem. Lots of projects and researches have been done to solve this problem. Nevertheless, they do not consider different users' conditions, only using a single consensus protocol as the best fit one, as well as the IoT system is heavily constrained by computing and networking resources. In this paper, we study a permissioned blockchain-based IoT architecture. In order to improve the scalability of the blockchain system and meet the needs of different users, we propose a service-oriented permissioned blockchain, where different consensus protocols are launched according to users' quality of service (QoS) requirements. Specially, we quantify a few popular consensus protocols. Additionally, we select block producers, which need a great number of computation resources, as well as dynamically allocate network bandwidth to the blockchain system. We formulate consensus protocols selection, block producers selection, and network bandwidth allocation as a joint optimization problem. We then use a dueling deep reinforcement learning approach to solve the problem. Simulation results demonstrate the effectiveness of our proposed scheme.

Index Terms—Internet of Things (IoT), permissioned blockchain, service-oriented, Byzantine fault tolerance, deep reinforcement learning.

I. INTRODUCTION

Recently, the Internet of Things (IoT) has stirred great interests from both industry and academia [1]. Similar to the traditional Internet that makes grand success to connect people, IoT has great potential to construct new relationships among devices and devices by ubiquitous connectivity, widespread sensors, and extensive accessibility. With the development of IoT, a sharp explosion in the number of IoT devices will happen in the next few years. It is anticipated that there will be 26 billion IoT devices in 2021 [2]. The era of *Internet of information* will shift to the era of *Internet of things*. Due to the expansion of the Internet and wireless networks, the development of wearable devices, and the cheaper values of embedded computers, a wide range of applications will be improved by IoT. Numerous innovative waves have happened in traditional applications, such as smart healthcare, smart grid, smart manufacturing, smart transportation, and smart agriculture. However, the

current IoT is heavily restricted by the highly centralized architecture, with the problems of single point of failure (SPOF), data privacy, reliability, robustness, etc.

Currently, blockchain has been regarded as one of the most reasonable candidates to address the above issues of IoT [3]. Blockchain works as a decentralized public ledger to store and share transactions, provides the implementation of decentralized applications, and manages the interactions in multi-peer systems. Blockchain relies on the utilization of cryptography, and individuals interact with others in a verifiable manner. Thus, it is natural to extend the success of blockchain from financial industry to IoT.

Nevertheless, numerous non-trivial problems in the current blockchain system prevent it from being used as a generic platform for large-scale services and applications in IoT. One of the most intractable drawbacks is the scalability problem [4]. For example, Bitcoin only has the capability of 7 transactions per second (TPS), while Ethereum has 15 TPS [5]. Their capabilities are far below the requirements of IoT, e.g., a midjet smart factory needs more than 5000 TPS capability, with a peak of 59,000 TPS [6]. The challenge has fueled the need to increase the throughput of blockchain, so as to improve scalability.

Recently, several projects and researches have been proposed to improve the scalability of blockchain, such as EOS [7], Zilliqa [8], and Plasma [9]. However, they do not consider different users' conditions, only using a single consensus protocol as the best fit one. For example, the underlying consensus protocol of EOS is democracy-as-proof-of-stake (dPoS), Practical Byzantine Fault Tolerance (PBFT) protocol powers Zilliqa, and Plasma uses proof-of-stake (PoS) protocol. In blockchain-enabled IoT, the quality of service (QoS) requirements vary significantly. For example, time-sensitive applications in IoT require low latency for the real-time delivery [10], while robustness is more important than latency when computation-intensive tasks are conducted. Therefore, it is unreasonable to use only one consensus protocol in the blockchain-enabled IoT with different users' QoS requirements.

In this paper, we study a blockchain-enabled IoT architecture, and propose a service-oriented permissioned blockchain for IoT. The contributions of this paper are summarized as follows.

- In order to improve the performance of the blockchain system and meet the needs of different users, we propose a service-oriented permissioned blockchain for IoT. Different consensus protocols are used to accommodate different users' QoS requirements.
- We quantify the performance of each consensus protocol, including throughput and latency, based on which the suitable consensus protocol is selected.
- In addition, the proposed blockchain can dynamically select block producers among IoT devices, and allocate network bandwidth.
- We formulate consensus protocol selection, block producer selection, and network bandwidth allocation as a joint optimization problem. We then use a dueling deep reinforcement learning approach to solve the problem.
- Simulation results with different system parameters are presented to show the effectiveness of the proposed scheme.

The rest of this paper is organized as follows. Section II presents the related work. Section III describes the system model, followed by the quantitative analyses of consensus protocols in Section IV. We then present the problem formulation in Section V. Simulation results are shown and discussed in Section VI. Finally, in Section VII, conclusions and future work are given.

II. RELATED WORK

In this section, we review some well-known BFT protocols, including PBFT, Zyzzyva, and Quorum. We then give a brief overview of the recent improvements about BFT in blockchain.

A. Well-Known BFT Protocols

As shown in [11], there are eight layers in the blockchain system, namely data layer, network layer, consensus layer, ledger topology layer, incentive layer, privacy layer, contract layer, and application layer. The BFT protocol is implemented at the consensus layer, which uses replication to maintain system resilience against Byzantine failures. The protocol enables the system to tolerate arbitrary failures in a bounded number of nodes, i.e., up to one in three BFT nodes is faulty.

PBFT is seen as the first practical BFT protocol. It runs a three-phase commit protocol, including *pre-prepare*, *prepare*, and *commit*, as shown in Fig. 1 [12]. Due to the fact that PBFT needs extensive messages processing and exchanges among all replicas, it enables to reach high system resilience against Byzantine failures, at the cost of the performance.

Zyzzyva is a speculative approach to achieve system resilience, improving the performance, and simplifying the design of state machine replication (SMR) protocol. Unlike PBFT running a complex three-phase commit protocol,

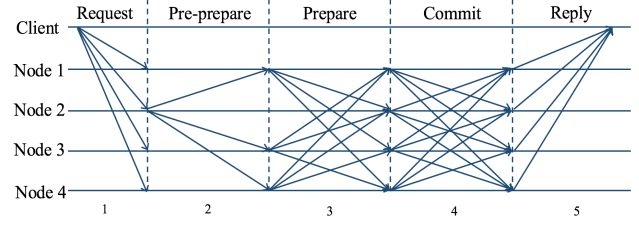


Fig. 1: The consensus pattern of PBFT protocol.

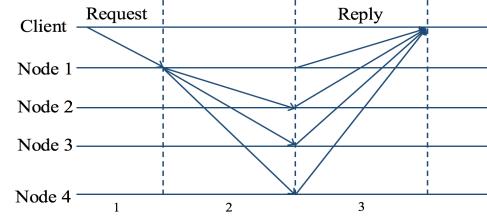


Fig. 2: The consensus pattern of best execution in Zyzzyva protocol.

Zyzzyva temporarily uses the results proposed by the *primary*, and immediately responds to the client. Then, the client detects faults and inconsistencies, as well triggers different consensus patterns. Thus, there are three different consensus patterns in Zyzzyva, including best execution, common faulty replica execution, and worst faulty replica execution, as shown in Fig. 2, Fig. 3, and Fig. 4, respectively [13].

Quorum is another speculative BFT protocol with the theoretical minimum latency. In a Byzantine fault-free system, it only implements one round trip message processing

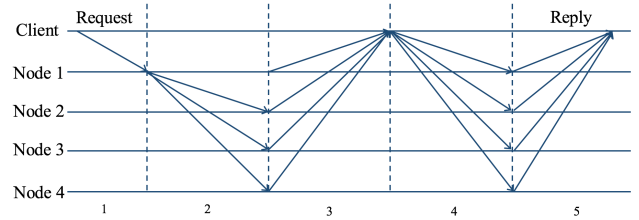


Fig. 3: The consensus pattern of common faulty replica execution in Zyzzyva protocol.

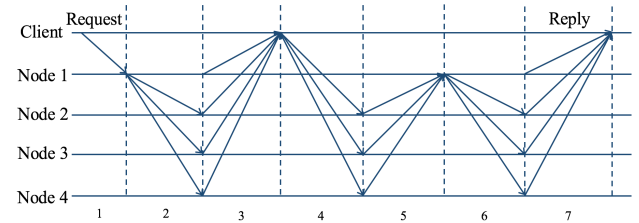


Fig. 4: The consensus pattern of worst faulty replica execution in Zyzzyva protocol.

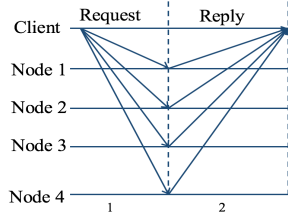


Fig. 5: The consensus pattern of best execution in Quorum protocol.

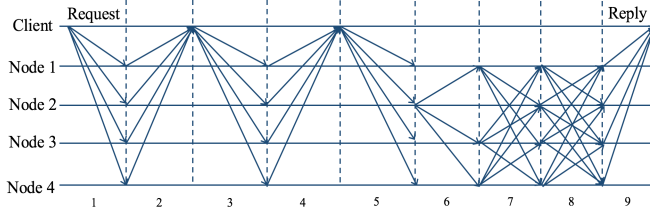


Fig. 6: The consensus pattern of other execution in Quorum protocol.

to reach consensus. Otherwise, it switches to the backup protocol. Thus, there are two different consensus patterns in Quorum, including best execution and other execution [14], as shown in Fig. 5, and Fig. 6, respectively.

B. Recent Improvements of BFT in Blockchain

Generally speaking, there are two kinds of blockchain, namely permissionless blockchain and permissioned blockchain. The popular utilization of working-based consensus protocol, such as PoW (Proof of Work), in permissionless blockchain needs compensation to agree on the validity of transactions [15]. It is disadvantageous for the system that operates without compensation. In addition, the traditional BFT has lots of fatal drawbacks, such as the completely enclosed environment, the limited scalability, the strong dependence on the *primary*, and the highly complicated messages exchanges.

Several excellent works have been proposed to address the above problems of BFT. In traditional BFT, the nodes who want to join in or leave the BFT system must stop the whole system. Thus, Xu *et al.* in [16] studied a dynamic BFT, which allowed nodes to join in and leave the BFT system dynamically. In order to improve the scalability of BFT, Liu *et al.* in [17] used a lightweight secret sharing based trusted execution hardware environment in the current BFT, called FastBFT. By doing this, FastBFT could have low latency and high throughput in large-scale systems. The work in [18] employed a dueling deep Q-learning approach to select a more trusted node as the *primary*. They also demonstrated that such PBFT can achieve higher throughput than the traditional PBFT. Aiming at the problem of complex messages exchanges, the authors in [19] presented a low-latency BFT. Here, remote direct memory access (RDMA)

was used in the multiple rounds of message exchanges in BFT. Compared with TCP, RDMA provided a kernel bypassing approach to relieve intermediate data copying.

As we can see, the above works only consider one kind of BFT protocol in the blockchain system. Clients' different consensus requirements and the scenarios for different consensus protocols are ignored. Fortunately, Bahsoun *et al.* in [20] conceived the idea of multiple SMR protocols, called ADAPT, which switched a running protocol to a new more appropriate one. Here, a quality control system, i.e., Key Characteristic Indicator (KCI) and Key Performance Indicator (KPI), was implemented to make decisions about the switching of SMR protocols. However, they did not quantify each SMR protocol, and some parameters were qualitative. Thus, in this paper, the quantitative analysis of each SMR protocol is presented. Based on these quantitative results, a service-oriented blockchain is proposed, switching different SMR protocols according to users' QoS requirements and system scenarios. Moreover, given the limited computation and bandwidth resources in IoT, we also make decisions about block producers and bandwidth allocation, so as to enhance the performance.

III. SYSTEM MODEL

In this section, we present the BFT fault model, the QoS model, the computation model, the bandwidth model, and the trust feature model.

A. BFT Fault Model

The BFT fault model is a fully connected and message-passing distributed system. We assume that there are N nodes in the permissioned blockchain, indexed by $\mathcal{N} = \{1, 2, \dots, n, \dots, N\}$. These N nodes are under BFT fault model, i.e., at most $f = \frac{N-1}{3}$ nodes are faulty to deliver and order messages. Additionally, we assume there are C users in IoT who want to use the permissioned blockchain, called clients in the following, and indexed by $\mathcal{C} = \{1, 2, \dots, c, \dots, C\}$. Any finite number of clients behave in an arbitrary way against the designed purposes. In addition, strong adversaries coordinate with each faulty node to compromise the replicated services. Nevertheless, they cannot violate cryptographic techniques, such as signatures, message authentication codes (MACs), and collision-resistant hashing.

B. QoS Model

Due to the fact that the service-oriented permissioned blockchain launches different consensus protocols according to QoS requirements, it is necessary to formulate the QoS requirements. We assume that there are P candidate consensus protocols, indexed by p , and there are two criteria to evaluate each protocol's service, namely throughput and latency. Thus, we can have the following matrix $\mathcal{Q}$, where

each row denotes each consensus protocol, the first column denotes the throughput criterion, while the second column denotes the latency criterion.

$$\mathcal{Q} = \begin{bmatrix} q_{1,1} & q_{1,2} \\ q_{2,1} & q_{2,2} \\ \vdots & \vdots \\ q_{p,1} & q_{p,2} \\ \vdots & \vdots \\ q_{P,1} & q_{P,2} \end{bmatrix}. \quad (1)$$

According to the work in [21], the matrix $\mathcal{Q}$ should be normalized by the following two steps:

(1) We define two arrays, including array $\mathcal{V}$ and array $\mathcal{M}$. Here, $\mathcal{V} = \{v_1, v_2\} = \{1, 0\}$. 1 indicates the increase of $q_{p,1}$, i.e., throughput, is beneficial to the protocol's service. 0 indicates the decrease of $q_{p,2}$, i.e., latency, is beneficial to the protocol's service. Additionally, $\mathcal{M} = \{m_1, m_2\}$. Here, m_1 and m_2 are two constants to set the maximum value of normalization. Thus, we use the following equations to normalize matrix $\mathcal{Q}$, where $1 \leq i \leq P$ and $j \in \{1, 2\}$.

- When $\frac{1}{P} \sum_{i=1}^P q_{i,j} \neq 0$, $\frac{q_{i,j}}{\frac{1}{P} \sum_{i=1}^P q_{i,j}} < m_j$, and $v_j = 1$, $q'_{i,j} = \frac{q_{i,j}}{\frac{1}{P} \sum_{i=1}^P q_{i,j}}$.
- When $\frac{1}{P} \sum_{i=1}^P q_{i,j} = 0$ and $v_j = 1$, or $\frac{q_{i,j}}{\frac{1}{P} \sum_{i=1}^P q_{i,j}} \geq m_j$, $q'_{i,j} = m_j$.
- When $q_{i,j} \neq 0$, $\frac{1}{P} \sum_{i=1}^P q_{i,j} < m_j$, and $v_j = 0$, $q'_{i,j} = \frac{\frac{1}{P} \sum_{i=1}^P q_{i,j}}{q_{i,j}}$.
- When $q_{i,j} = 0$, and $v_j = 0$, or $\frac{1}{P} \sum_{i=1}^P q_{i,j} \geq m_j$, $q'_{i,j} = m_j$.

After applying the normalization to matrix $\mathcal{Q}$, we have $\mathcal{Q}'$ as follows:

$$\mathcal{Q}' = \begin{bmatrix} q'_{1,1} & q'_{1,2} \\ q'_{2,1} & q'_{2,2} \\ \vdots & \vdots \\ q'_{p,1} & q'_{p,2} \\ \vdots & \vdots \\ q'_{P,1} & q'_{P,2} \end{bmatrix}. \quad (2)$$

For example, there are three candidate consensus protocols, and we have matrix $\mathcal{Q} = \{q_{1,1}, q_{1,2}; q_{2,1}, q_{2,2}; q_{3,1}, q_{3,2}\} = \{1509.4, 80; 2051.3, 48; 2424.2, 32\}$. We set $\mathcal{M} = \{m_1, m_2\} = \{5, 5\}$. Using the above equations, we have matrix $\mathcal{Q}' = \{0.757, 0.667; 1.028, 1.111; 1.215, 1.667\}$.

(2) We define user's preference array $F = \{f_1, f_2\}$, where f_1 and f_2 mean the weights of user's preferences, such as throughput sensitivity and latency sensitivity. Given the uncertainty and the time correlation of users' preferences,

we model the user's throughput preference and latency preference as random variables f_1 and f_2 . We then use Markov chains to model the transition of them, whose transition probability matrices are $\mathcal{F}_1(t)$ and $\mathcal{F}_2(t)$, where $t \in \{0, 1, 2, \dots, T-1\}$ denotes the time instant.

Finally, we obtain QoS value for each candidate consensus protocol with array F and matrix $\mathcal{Q}'$. The equation of QoS value is shown as follows:

$$QoS(p) = q'_{p,1} * f_1 + q'_{p,2} * f_2. \quad (3)$$

For example, we obtained $\mathcal{Q}' = \{0.757, 0.667; 1.028, 1.111; 1.215, 1.667\}$. At this time slot, user's preference array $F = \{1, 1\}$. Thus, we have $QoS(1) = 1.424$, $QoS(2) = 2.139$, and $QoS(3) = 2.882$.

C. Computation Model

The nodes in IoT usually have limited computation resources. Thus, it is necessary to select a block producer, who has a great number of computation resources. There are various tasks in the IoT system, and we do not exactly know the computation capability of node n at the next time slot. Therefore, we can model the computation capability of node n as a random variable ϑ^n . Assuming the value of computation capability can be partitioned into X discrete intervals, denoted as $\mathcal{X} = \{\mathcal{X}_0, \mathcal{X}_1, \dots, \mathcal{X}_{X-1}\}$. The computation capability of node n at time slot t can be expressed as $\vartheta^n(t)$. Considering the time correlation of computation states in nodes, we then model the transition of computation capability as a Markov chain. Thus, the $X \times X$ transition probability matrix $\Theta^n(t)$ can be expressed as follows:

$$\Theta^n(t) = [Pr(\vartheta^n(t+1) = l_s | \vartheta^n(t) = w_s)]_{X \times X}, \quad (4)$$

where $l_s, w_s \in \mathcal{X}$.

D. Bandwidth Model

Similarly, the IoT system is also network bandwidth resource-limited, and a great number of cryptographic messages are delivered inside the IoT system. Therefore, we allocate network bandwidth resources dynamically to the system, so as to improve the performance.

Assuming there are B links are available, we cannot exactly know how many bandwidth resources are accessible at the next time instant. Thus, the bandwidth resources can be modeled as a random variable v^b , which can be divided into R discrete intervals, shown as $\mathcal{R} = \{\mathcal{R}_0, \mathcal{R}_1, \dots, \mathcal{R}_{R-1}\}$. Let $v^b(t)$ represent the accessible bandwidth resources at time slot t . Based on the time correlation of bandwidth resources, we use a Markov chain to model the transition of random variable $v^b(t)$. Based on $R \times R$ transition probability matrix $\Upsilon^b(t)$, the accessible bandwidth resources change from one state to another, which can be denoted as:

$$\Upsilon^b(t) = [Pr(v^b(t+1) = b_s | v^b(t) = a_s)]_{R \times R}, \quad (5)$$

where $a_s, b_s \in \mathcal{R}$.

E. Trust Model

We consider the trust features of the IoT system. Due to the lack of centralized security management, the IoT system has diverse trust features, such as secure and compromised. It is challenging to exactly know what the trust is for the IoT system at the next instant. Thus, we model the trust of the IoT system as a random variable ϱ , where ϱ can be divided into Z discrete levels, denoted by $\zeta = \{\zeta_0, \zeta_1, \dots, \zeta_{Z-1}\}$. The trust of ϱ at time instant t can be denoted as $\varrho(t)$, where $t \in \{0, 1, 2, \dots, T-1\}$. Considering the time correlation of trust features in the IoT system, we use a Markov chain to model the transition of trust features. Thus, the $Z \times Z$ transition probability matrix $\mathcal{Z}(t)$ of the trust features in the IoT system is denoted as

$$\mathcal{Z}(t) = [Pr(\varrho(t+1) = m_s | \varrho(t) = n_s)]_{Z \times Z}, \quad (6)$$

where $m_s, n_s \in \zeta$.

IV. QUANTITATIVE ANALYSES OF CONSENSUS PROTOCOLS

After presenting the system model, we then give the quantitative analysis of each consensus protocol. Here, we mainly consider three typical consensus protocols, PBFT, Zyzzyva, and Quorum. Additionally, we elaborate on quantitative analysis of throughput and latency in each consensus protocol. Meanwhile, the analytical methodology can be extended to other consensus protocols.

In this section, we assume that verifying one signature needs θ CPU cycles, verifying and generating one MAC need α CPU cycles, verifying smart contract needs β CPU cycles, one block has b transactions, each IoT node has μ Hz computation speed, and the blockchain system has $\mathcal{B}$ Mbps bandwidth resources.

A. Quantitative Analysis of PBFT

We have presented the quantitative analysis of PBFT in our previous work [18], [22]. Here, we give a brief review and modification.

As shown in Fig. 1, there are five steps in PBFT. The numbering of each step in this figure is the same as the one used in the remainder of this subsection.

1. The client sends an un-validated block to all nodes.

The client sends an un-validated block, including b transactions, to all nodes. Based on a certain view change protocol [12], one node is selected as the *primary* node. Other nodes are replica nodes. Only the *primary* verifies MACs and signatures of the un-validated block and b transactions.

According [23], the required time of one single peer to deliver the un-validated block can be denoted as $t_{hop} = b/\mathcal{B}$. The *primary* verifies $(b+1)$ signatures and $(b+1)$ MACs, where b MACs and b signatures from b transactions, and one MAC and one signature from the un-validated block.

2. **The *primary* sends PRE-PREPARE messages to all replicas.** In this stage, the *primary* generates $(N-1)$ MACs. Replicas verify one MAC from the *primary*, b MACs and b signatures from b transactions. Totally, the *primary*'s computation cost is $(N-1)\alpha$, and replicas' computation costs are $\alpha + b(\theta + \alpha)$. Similarly, the time of one single peer is still needed to deliver messages, i.e., $t_{hop} = b/\mathcal{B}$.

3. **Replicas send PREPARE messages to all others.** Replicas generate $(N-1)$ MACs, and send them to all others. After receiving $2f$ matching PREPARE messages with the local PRE-PREPARE message, the nodes enter the following steps. Thus, each replica's computation cost is $(N-1+2f)\alpha$, and the *primary*'s computation cost is $2f\alpha$. Meanwhile, the time delay of messages delivery is $t_{hop} = b/\mathcal{B}$.

4. **COMMIT messages from all nodes are sent to all others.** All nodes generate $(N-1)$ MACs, and send them to all others. Similarly, after receiving $2f$ matching COMMIT messages with the local PREPARE message, the nodes verify b smart contracts from b transactions. Therefore, all nodes' computation costs are $(N-1+2f)\alpha + b\beta$, and the time delay of message delivery is $t_{hop} = b/\mathcal{B}$.

5. **All nodes send the validated block to all clients.** All nodes reply the validated block to all clients. Thus, all nodes generate one MAC, with α computation costs.

In summary, for b transactions, the total computation costs of the *primary* and replicas are shown in Table I. Totally, the computation cost of the *primary* for one transaction is

$$\eta_{PBFT_p} = \frac{(b+1)(\theta + \alpha) + (2N+4f-1)\alpha + b\beta}{b}. \quad (7)$$

The computation cost of a replica for one transaction is

$$\eta_{PBFT_r} = \frac{b(\theta + \alpha) + (2N+4f)\alpha + b\beta}{b}. \quad (8)$$

The computation speed of each node is μ Hz. Thus, the throughput of PBFT consensus protocol is at most

$$\begin{aligned} \tau_{PBFT} &= \frac{\mu}{\max(\eta_{PBFT_p}, \eta_{PBFT_r})} \\ &= \frac{\mu}{\frac{(b+1)(\theta + \alpha) + (2N+4f-1)\alpha + b\beta}{b}}. \end{aligned} \quad (9)$$

Due to the fact that there are five delivery steps in PBFT consensus protocol, the latency of PBFT is

$$\iota_{PBFT} = 5 * t_{hop} = \frac{5b}{\mathcal{B}}. \quad (10)$$

B. Quantitative Analysis of Zyzzyva

According to the work in [13], Zyzzyva is a speculative consensus protocol, in order to relieve the computation cost and simplify the implementation of BFT protocol. Zyzzyva enables to speed up the consensus system as long as the client receives matching messages from all nodes. Otherwise, complicated recovery steps are launched when

TABLE I: The computation cost of each node in PBFT

Step	Computation cost
Step 1	The <i>primary</i> : $(b+1)(\theta + \alpha)$
	A replica: 0
Step 2	The <i>primary</i> : $(N-1)\alpha$
	A replica: $\alpha + b(\theta + \alpha)$
Step 3	The <i>primary</i> : $2f\alpha$
	A replica: $(N-1+2f)\alpha$
Step 4	The <i>primary</i> : $(N-1+2f)\alpha + b\beta$
	A replica: $(N-1+2f)\alpha + b\beta$
Step 5	The <i>primary</i> : α
	A replica: α

the *primary* and replicas are faulty. Thus, there are three consensus patterns in Zyzyva, including best execution, common faulty replica execution, and worst faulty replica execution.

Best execution in Zyzyva:

As shown in Fig. 2, there are three steps in best execution of Zyzyva. The numbering of each step in this figure is the same as the one used in the best execution of Zyzyva.

1. **The client sends an un-validated block to the *primary*.** In a view, a node is assigned as the *primary*. The *primary* verifies one MAC and one signature from the un-validated block. If valid, it will enter the following step. Thus, the computation cost of the *primary* is $(\theta + \alpha)$.

2. **The *primary* verifies and sends transactions to all replicas.** The *primary* verifies b MACs and b signatures from b transactions. Then it generates $(N-1)$ MACs and sends transactions to all replicas. Thus, the computation cost of the *primary* is $b(\theta + \alpha) + (N-1)\alpha$.

3. **Replicas verify transactions, all nodes execute smart contracts, and reply the client.** Replicas receive transactions from the *primary*, and verify MACs and signatures. Due to the fact that Zyzyva is a speculative consensus protocol, all nodes execute smart contracts before its validity is fully verified. So they execute smart contracts, generate a validated block, and reply the client. Thus, the computation cost of the *primary* is $b\beta + \alpha$, and the computation cost of a replica is $2\alpha + b(\theta + \alpha) + b\beta$.

The validated block completes at the client in one of three ways.

First, in the best execution of Zyzyva, the client receives all blocks that are consistent with each other, i.e., $N = 3f + 1$ consistent blocks, it accepts this block and appends it in its chain. Thus, for b transactions, the total computation costs of the *primary*, replicas, and the client are shown in Table II. Totally, the computation cost of the *primary* for one transaction is

$$\eta_{ZB_p} = \frac{(b+1)(\theta + \alpha) + N\alpha + b\beta}{b}. \quad (11)$$

The computation cost of a replica for one transaction is

$$\eta_{ZB_r} = \frac{2\alpha + b(\theta + \alpha) + b\beta}{b}. \quad (12)$$

TABLE II: The computation cost of each node in best execution of Zyzyva

Step	Computation cost
Step 1	The <i>primary</i> : $\theta + \alpha$
	A replica: 0
Step 2	The <i>primary</i> : $b(\theta + \alpha) + (N-1)\alpha$
	A replica: 0
Step 3	The <i>primary</i> : $b\beta + \alpha$
	A replica: $2\alpha + b(\theta + \alpha) + b\beta$
	The client: $(3f+1)\alpha$

The computation cost of a client for one transaction is

$$\eta_{ZB_c} = \frac{(3f+1)\alpha}{b}. \quad (13)$$

The computation speed of each node is μ Hz. Thus, the throughput of best execution in Zyzyva is at most

$$\begin{aligned} \tau_{ZB} &= \frac{\mu}{\max(\eta_{ZB_p}, \eta_{ZB_r}, \eta_{ZB_c})} \\ &= \frac{\mu}{\max(\frac{(b+1)(\theta + \alpha) + N\alpha + b\beta}{b}, \frac{3f+1}{b})}. \end{aligned} \quad (14)$$

Due to the fact that there are three delivery steps in best execution of Zyzyva, its latency is

$$\iota_{ZB} = 3 * t_{hop} = \frac{3b}{B}. \quad (15)$$

Second, the client only receives between $2f + 1$ and $3f$ consistent blocks, which is the second pattern in Zyzyva, i.e., common faulty replica execution.

Common faulty replica execution in Zyzyva:

As shown in Fig. 3, there are five steps in common faulty replica execution of Zyzyva. The first three steps are the same as those in best execution of Zyzyva. Let's start from step 4.

4. **The client sends a commit certificate message to all nodes.** If the *primary* and replicas are faulty, the client will never receive $3f + 1$ consistent blocks. If receiving $2f + 1$ consistent blocks, the client sends a commit certificate message to all nodes. Thus, the computation cost of the client is $(2f + 1)\alpha + N\alpha$.

5. **All nodes send a LOCAL COMMIT message to the client.** All nodes receive and verify a commit certificate from the client, proving that the block should be ordered with a new sequence number in the current view. Then all nodes send a LOCAL COMMIT message to the client. If receiving $2f + 1$ consistent LOCAL COMMIT messages from nodes, the client accepts this block, and appends it in its chain. Thus, the computation cost of each node is 2α , and the computation cost of the client is $(2f + 1)\alpha$.

Thus, for b transactions, the total computation costs of the *primary*, replicas, and the client are shown in Table III. The computation cost of the *primary* for one transaction is

$$\eta_{ZC_p} = \frac{(b+1)(\theta + \alpha) + (N+2)\alpha + b\beta}{b}. \quad (16)$$

TABLE III: The computation cost of each node in common faulty replica execution of Zyzzyva

Step	Computation cost
Step 1	The <i>primary</i> : $\theta + \alpha$
	A replica: 0
Step 2	The <i>primary</i> : $b(\theta + \alpha) + (N - 1)\alpha$
	A replica: 0
Step 3	The <i>primary</i> : $b\beta + \alpha$
	A replica: $2\alpha + b(\theta + \alpha) + b\beta$
Step 4	The <i>primary</i> : 0
	A replica: 0
	The client: $(2f + 1 + N)\alpha$
Step 5	The <i>primary</i> : 2α
	A replica: 2α
	The client: $(2f + 1)\alpha$

The computation cost of a replica for one transaction is

$$\eta_{ZC_r} = \frac{b(\theta + \alpha) + b\beta + 4\alpha}{b}. \quad (17)$$

The computation cost of the client for one transaction is

$$\eta_{ZC_c} = \frac{(4f + N + 2)\alpha}{b}. \quad (18)$$

The computation speed of each node is μ Hz. Thus, the throughput of common faulty replica execution in Zyzzyva is at most

$$\begin{aligned} \tau_{ZC} &= \frac{\mu}{\max(\eta_{ZC_p}, \eta_{ZC_r}, \eta_{ZC_c})} \\ &= \frac{\mu}{\max(\frac{(b+1)(\theta+\alpha)+(N+2)\alpha+b\beta}{b}, \frac{(4f+N+2)\alpha}{b})}. \end{aligned} \quad (19)$$

Due to the fact that there are five delivery steps in common faulty replica execution of Zyzzyva, its latency is

$$\iota_{ZC} = 5 * t_{hop} = \frac{5b}{B}. \quad (20)$$

Third, the client receives fewer than $2f + 1$ consistent blocks, which is the third pattern in Zyzzyva, i.e., worst faulty replica execution.

Worst faulty replica execution in Zyzzyva:

As shown in Fig. 4, there are seven steps in worst faulty replica execution of Zyzzyva. The first three steps are the same as those in best execution of Zyzzyva. Let's start from step 4.

4. The client receives fewer than $2f + 1$ consistent blocks, and resends an un-validated block to all replicas. If the *primary* or replicas are badly compromised by strong adversaries, the client will never receive $2f + 1$ consistent blocks. Thus, it resends the un-validated block, including b transactions, to all replicas. Thus, the client firstly verifies $2f + 1$ MACs, then generates $N - 1$ MACs, whose computation costs are $(2f + N)\alpha$.

5. Replicas send a confirm request message to the *primary*. Replicas receive the un-validated block from the client, verifying one MAC from the client. Then they send a confirm request message to the current *primary*, where each

replica generates one MAC, and sends to the *primary*. Thus, the computation costs of replicas are 2α .

6. The *primary* receives confirm request messages from replicas. The *primary* receives and verifies confirm request messages from replicas. If the request is not new, it sends transactions to all replicas. Thus, the computation costs of the *primary* are $2(N - 1)\alpha$.

7. Replicas verify transactions, all nodes execute smart contracts, and reply the client. Replicas receive transactions from the *primary*, and verify one MAC from the *primary*. If valid, the *primary* and replicas execute smart contracts. Then they send the validated block to the client, generating one MAC. Finally, the client receives the validated block and verifies their MACs. Thus, the computation costs of the *primary*, replicas, and the client are $b\beta + \alpha$, $b\beta + 2\alpha$, and $(3f + 1)\alpha$, respectively.

Thus, for b transactions, the total computation costs of the *primary*, replicas, and the client are shown in Table IV. The computation cost of the *primary* for one transaction is

$$\eta_{ZW_p} = \frac{(b + 1)(\theta + \alpha) + 2b\beta + (3N - 1)\alpha}{b}. \quad (21)$$

The computation cost of a replica for one transaction is

$$\eta_{ZW_r} = \frac{b(\theta + \alpha) + 2b\beta + 6\alpha}{b}. \quad (22)$$

The computation cost of the client for one transaction is

$$\eta_{ZW_c} = \frac{(N + 5f + 1)\alpha}{b}. \quad (23)$$

The computation speed of each node is μ Hz. Thus, the throughput of worst faulty replica execution in Zyzzyva is at most

$$\begin{aligned} \tau_{ZW} &= \frac{\mu}{\max(\eta_{ZW_p}, \eta_{ZW_r}, \eta_{ZW_c})} \\ &= \frac{\mu}{\max(\frac{(b+1)(\theta+\alpha)+2b\beta+(3N-1)\alpha}{b}, \frac{(N+5f+1)\alpha}{b})}. \end{aligned} \quad (24)$$

There are seven steps in worst faulty replica execution of Zyzzyva, so its latency is

$$\iota_{ZW} = 7 * t_{hop} = \frac{7b}{B}. \quad (25)$$

C. Quantitative Analysis of Quorum

Based on the work in [14], when there are no link failures, Byzantine fault-free, and no contention, Quorum has a very simple consensus pattern. Otherwise, backup consensus protocol will be implemented. Therefore, there are two different consensus patterns in Quorum, including best execution and other execution.

Best execution in Quorum:

As shown in Fig. 5, there are two steps in best execution of Quorum. The numbering of each step in this figure is the same as the one used in the best execution of Quorum.

TABLE IV: The computation cost of each node in worst faulty replica execution of Zyzzyva

Step	Computation cost
Step 1	The <i>primary</i> : $\theta + \alpha$
	A replica: 0
Step 2	The <i>primary</i> : $b(\theta + \alpha) + (N - 1)\alpha$
	A replica: 0
Step 3	The <i>primary</i> : $b\beta + \alpha$
	A replica: $2\alpha + b(\theta + \alpha) + b\beta$
Step 4	The <i>primary</i> : 0
	A replica: 0
Step 5	The client: $(2f + N)\alpha$
	The <i>primary</i> : 0
Step 6	A replica: 2α
	The client: 0
Step 7	The <i>primary</i> : $2(N - 1)\alpha$
	A replica: 0
Step 8	The client: 0
	The <i>primary</i> : $b\beta + \alpha$
Step 9	A replica: $b\beta + 2\alpha$
	The client: $(3f + 1)\alpha$

1. The client sends an un-validated block to all nodes.

The client sends an un-validated block to all nodes, including b transactions. All nodes verify one MAC and one signature from the client, and b MACs and b signatures from transactions. If valid, it will enter the following steps. Thus, the computation costs of all nodes are $(b + 1)(\theta + \alpha)$.

2. All nodes execute smart contracts and reply the client. All nodes execute smart contracts, generate a validated block, and reply, before the validity of this un-validated block is fully verified. Thus, the computation costs of all nodes are $b\beta + \alpha$.

This block completes at the client in one of two ways.

First, in the best execution of Quorum, the client receives $3f + 1$ consistent blocks, it accepts this block and appends it in its own chain, where the client needs to verify $3f + 1$ MACs from all nodes. Thus, for b transactions, the total computation costs of each node and the client are shown in Table V. The computation cost of each node for one transaction is

$$\eta_{QB_p} = \frac{(b + 1)(\theta + \alpha) + b\beta + \alpha}{b}. \quad (26)$$

The computation cost of the client for one transaction is

$$\eta_{QB_c} = \frac{(3f + 1)\alpha}{b}. \quad (27)$$

The computation speed of each node is μ Hz. Thus, the throughput of best execution in Quorum is at most

$$\tau_{QB} = \frac{\mu}{\max(\eta_{QB_p}, \eta_{QB_r}, \eta_{QB_c})} = \frac{\mu}{\max(\frac{(b+1)(\theta+\alpha)+b\beta+\alpha}{b}, \frac{(3f+1)\alpha}{b})}. \quad (28)$$

There are two steps in best execution of Quorum, and its latency is

$$\iota_{QB} = 2 * t_{hop} = \frac{2b}{B}. \quad (29)$$

TABLE V: The computation cost of each node in best execution of Quorum

Step	Computation cost
Step 1	All nodes: $(b + 1)(\theta + \alpha)$
Step 2	All nodes: $b\beta + \alpha$
	The client: $(3f + 1)\alpha$

Second, the client receives fewer than $3f + 1$ consistent blocks, which is the second pattern in Quorum, i.e., other execution.

Other execution in Quorum:

As shown in Fig. 6, there are nine steps in other execution of Quorum. The first two steps are the same as those in best execution of Quorum. Let's start from step 3.

3. The client receives fewer than $3f + 1$ blocks, and sends PANIC messages to all nodes. If some nodes are faulty, the client will never receive $3f + 1$ consistent blocks. When receiving fewer than $3f + 1$ consistent blocks, the client enters a panic pattern, sending PANIC messages to all nodes. Thus, the computation cost of the client is $(2f + N + 1)\alpha$.

4. Each node receives a PANIC message, and sends back a historical message. Each node receives a PANIC message from the client, verifying one MAC from the client. Then it sends back a historical message to the client, generating one MAC. When the client receives $2f + 1$ historical messages, it will enter the following step. Thus, the computation costs of each node and the client are 2α and $(2f + 1)\alpha$, respectively.

5. The client receives historical messages, and a backup consensus protocol is implemented. The client receives $2f + 1$ historical messages, knowing that the consensus progress is aborted. Then it resend the un-validated block, and a backup consensus protocol is implemented. In this article, we assume PBFT consensus protocol is the backup protocol. Thus, PBFT protocol is implemented from step 5.

Thus, for b transactions, the total computation costs of each node and the client are shown in Table VI. Thus, the computation cost of the *primary* for one transaction is

$$\eta_{QO_p} = \frac{2(b + 1)(\theta + \alpha) + 2b\beta + (2N + 4f + 2)\alpha}{b}. \quad (30)$$

The computation cost of a replica for one transaction is

$$\eta_{QO_r} = \frac{(2b + 1)(\theta + \alpha) + 2b\beta + (2N + 4f + 3)\alpha}{b}. \quad (31)$$

The computation cost of the client for one transaction is

$$\eta_{QO_c} = \frac{(4f + N + 2)\alpha}{b}. \quad (32)$$

TABLE VI: The computation cost of each node in other execution of Quorum

Step	Computation cost
Step 1	All nodes: $(b+1)(\theta+\alpha)$
Step 2	All nodes: $b\beta+\alpha$
Step 3	All nodes: 0 The client: $(2f+N+1)\alpha$
Step 4	All nodes: 2α The client: $(2f+1)\alpha$
Step 5	The primary: $(b+1)(\theta+\alpha)$ A replica: 0
Step 6	The primary: $(N-1)\alpha$ A replica: $\alpha+b(\theta+\alpha)$
Step 7	The primary: $2f\alpha$ A replica: $(N-1+2f)\alpha$
Step 8	The primary: $(N-1+2f)\alpha+b\beta$ A replica: $(N-1+2f)\alpha+b\beta$
Step 9	The primary: α A replica: α

The computation speed of each node is μ Hz. Thus, the throughput of other execution in Quorum is at most

$$\tau_{QO} = \frac{\mu}{\max(\eta_{QO_p}, \eta_{QO_r}, \eta_{QO_c})} = \frac{\mu}{\max(\frac{2(b+1)(\theta+\alpha)+2b\beta+(2N+4f+2)\alpha}{b}, \frac{(4f+N+2)}{b})}. \quad (33)$$

There are nine steps in other execution of Quorum, and its latency is

$$\iota_{QO} = 9 * t_{hop} = \frac{9b}{B}. \quad (34)$$

So far, We have finished quantitative analyses of the consensus protocols, including PBFT, Zyzzyva, and Quorum. For a comprehensive perspective, Table VII offers the quantitative results of consensus protocols.

V. PROBLEM FORMULATION

We have presented the quantitative analyses results of consensus protocols. In order to achieve a service-oriented permissioned blockchain and improve the performance of the blockchain system, it is necessary to make joint decisions about the selection of protocol candidates, block producers, and network bandwidth, which is a joint optimization problem. In this section, we use a dueling deep reinforcement learning approach to solve this problem. We firstly define system state, system action, and reward function to formulate this problem as a Markov decision process. Then the dueling deep reinforcement learning approach is presented.

A. Markov Decision Process

In order to formulate the joint problem as a Markov decision process, it is necessary to re-define system state, system action, and reward function [24].

1) *System State*: The learning agent senses system state $S(t)$ at time instant t . Due to the fact that the learning agent makes decisions about the selection of protocol candidates, block producers, and network bandwidth, system state can be expressed as follows:

$$S(t) = \begin{bmatrix} \varrho(t) & f_1(t) & f_2(t) \\ \vartheta^1(t) & \vartheta^2(t) & \dots & \vartheta^N(t) \\ v^1(t) & v^2(t) & \dots & v^B(t) \end{bmatrix}. \quad (35)$$

2) *System Action*: The learning agent makes decisions about protocol candidates (i.e., which consensus protocol is suitable under the client's preference and trust feature of the IoT system), block producer (i.e., which IoT node produces the block), and network bandwidth (i.e., which bandwidth is allocated to the IoT system). Thus, system action can be denoted as follows:

$$\mathcal{A}(t) = \{\mathcal{A}^P(t), \mathcal{A}^N(t), \mathcal{A}^B(t)\}, \quad (36)$$

where

- $\mathcal{A}^P(t) = \{a^1(t), a^2(t), \dots, a^p(t), \dots, a^P(t)\}$ means which candidate consensus protocol is implemented. Here, $a^p(t) \in \{0, 1\}$, and $a^p(t) = 1$ denotes protocol p is implemented, otherwise $a^p(t) = 0$. There is only one consensus protocol used in the blockchain system at one time, and $\sum_{p=1}^P a^p(t) = 1$.
- $\mathcal{A}^N(t) = \{a^1(t), a^2(t), \dots, a^n(t), \dots, a^N(t)\}$ means which node is the block producer. Here, $a^n(t) \in \{0, 1\}$, and $a^n(t) = 1$ denotes that node n is selected as a block producer, otherwise $a^n(t) = 0$. There is only one block producer in the blockchain system, and $\sum_{n=1}^N a^n(t) = 1$.
- $\mathcal{A}^B(t) = \{a^1(t), a^2(t), \dots, a^b(t), \dots, a^B(t)\}$ means which bandwidth can be allocated to the IoT system. Here, $a^b(t) \in \{0, 1\}$, and $a^b(t) = 1$ denotes that bandwidth b is allocated to the IoT system, otherwise $a^b(t) = 0$. There is only one bandwidth used at one time slot, and $\sum_{b=1}^B a^b(t) = 1$.

3) *Reward Function*: In order to achieve a service-oriented blockchain, our target is to maximize the QoS of the client. Thus, we model the QoS of the client as reward function. For example, the current trust feature of the IoT system is secure, and there are three candidate consensus protocols, including PBFT, Zyzzyva, and Quorum. Thus, according to Table VII, system state, and system action, we have matrix $\mathcal{Q}$ as follows:

$$\mathcal{Q} = \begin{bmatrix} \frac{\sum_{n=1}^N a^n(t) \vartheta^n(t)}{\max(\frac{(b+1)(\theta+\alpha)+(2N+4f-1)\alpha+b\beta}{b}, \frac{3f+1}{b})} & \frac{\sum_{b=1}^B 5b a^b(t) v^b(t)}{\sum_{b=1}^B 3b a^b(t) v^b(t)} \\ \frac{\sum_{n=1}^N a^n(t) \vartheta^n(t)}{\max(\frac{(b+1)(\theta+\alpha)+N\alpha+b\beta}{b}, \frac{3f+1}{b})} & \frac{\sum_{b=1}^B 2b a^b(t) v^b(t)}{\sum_{b=1}^B 2b a^b(t) v^b(t)} \end{bmatrix}. \quad (37)$$

TABLE VII: The quantitative analysis results of consensus protocols

Consensus protocol	Throughput	Latency
PBFT	$\tau_{PBFT} = \frac{\mu}{(b+1)(\theta+\alpha) + \frac{(2N+4f-1)\alpha+b\beta}{b}}$	$\frac{5b}{B}$
Zyzyva	Best execution: $\tau_{ZB} = \frac{\mu}{\max(\frac{(b+1)(\theta+\alpha)+N\alpha+b\beta}{b}, \frac{(3f+1)\alpha}{b})}$	Best execution: $\iota_{ZB} = \frac{3b}{B}$
	Common faulty replica execution: $\tau_{ZC} = \frac{\mu}{\max(\frac{(b+1)(\theta+\alpha)+(N+2)\alpha+b\beta}{b}, \frac{(4f+N+2)\alpha}{b})}$	Common faulty replica execution: $\iota_{ZC} = \frac{5b}{B}$
	Worst faulty replica execution: $\tau_{ZW} = \frac{\mu}{\max(\frac{(b+1)(\theta+\alpha)+2b\beta+(3N-1)\alpha}{b}, \frac{(N+5f+1)\alpha}{b})}$	Worst faulty replica execution: $\iota_{ZW} = \frac{7b}{B}$
Quorum	Best execution: $\tau_{QB} = \frac{\mu}{\max(\frac{(b+1)(\theta+\alpha)+b\beta+\alpha}{b}, \frac{(3f+1)\alpha}{b})}$	Best execution: $\iota_{QB} = \frac{2b}{B}$
	Other execution : $\tau_{QO} = \frac{\mu}{\max(\frac{2(b+1)(\theta+\alpha)+2b\beta+(2N+4f+2)\alpha}{b}, \frac{(4f+N+2)\alpha}{b})}$	Other execution: $\iota_{QO} = \frac{9b}{B}$

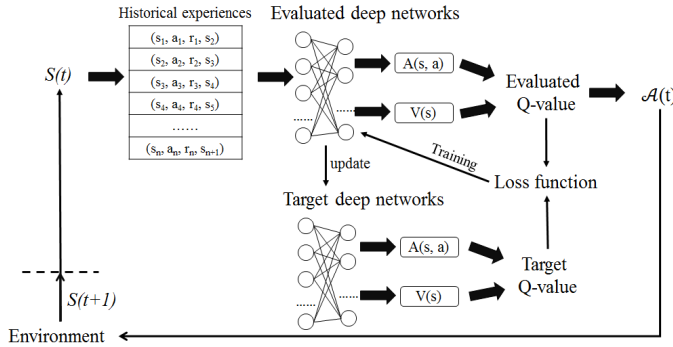


Fig. 7: The workflow of dueling DQL .

Then normalize matrix Q as Q' , which is shown in Section III.B. Using $f_1(t)$ and $f_2(t)$, we can obtain the QoS of the current client.

B. Dueling Deep Reinforcement Learning

After formulating the joint problem as a Markov decision process, we use a dueling deep reinforcement learning approach to solve this problem. To be specific, it is a deep Q-learning (DQL) approach.

We begin with DQL. It is an improved version of Q-learning that uses a Q-table to store action-state value function $Q(s, a)$. However, with the explosion of data dimension, it is barely possible to store all $Q(s, a)$ into Q-table. Thus, DQL uses deep networks to approximate $Q(s, a)$, so as to solve the challenge. This is the core idea of DQL.

In our system, the choice of an action has no relationship with the next system state. Dueling DQL is more efficient than natural DQL [25]. There are two separate outputs from deep networks in dueling DQL, one approximating state value function $V(s)$, another approximating state-action value function $A(s, a)$. Learning $A(s, a)$ is easier to know which action has better consequence. Fig. 7 shows the workflow of dueling DQL.

VI. SIMULATION RESULTS AND DISCUSSIONS

In this section, we use computer simulation to evaluate the numerical results of our proposed scheme. We firstly give simulation settings. Then simulation results are shown and discussed.

A. Simulation Settings

We present our performance results characterizing the proposed service-oriented blockchain in the context of IoT system considered. Moreover, we use Python 3.6.6 with TensorFlow 1.4.0, and Matlab simulations to evaluate our proposed scheme. We assume that there are three candidate consensus protocols, namely PBFT, Zyzyva, and Quorum. Seven IoT nodes join in the permissioned blockchain. Without loss of generality, we also assume that there are five available links to provide bandwidth resources to the IoT system.

The user's throughput preference and latency preference are high, medium, and low. We set their transition probability matrices as

$$\mathcal{F}_1 = \begin{bmatrix} 0.55 & 0.35 & 0.1 \\ 0.35 & 0.55 & 0.1 \\ 0.1 & 0.35 & 0.55 \end{bmatrix}. \quad (38)$$

$$\mathcal{F}_2 = \begin{bmatrix} 0.45 & 0.35 & 0.2 \\ 0.35 & 0.45 & 0.2 \\ 0.2 & 0.35 & 0.45 \end{bmatrix}. \quad (39)$$

The computation capability of each IoT node is formulated as a Markov model. Assuming that the state of the computation capability can be very high, high, medium, and low, whose transition probability matrix is set as

$$\Theta = \begin{bmatrix} 0.5 & 0.3 & 0.15 & 0.05 \\ 0.3 & 0.5 & 0.15 & 0.05 \\ 0.15 & 0.3 & 0.5 & 0.05 \\ 0.15 & 0.3 & 0.05 & 0.5 \end{bmatrix}. \quad (40)$$

TABLE VIII: The differences of these comparison schemes.

Item	Dynamic consensus protocols	Dynamic block producer	Dynamic network bandwidth
The proposed scheme	✓	✓	✓
The proposed scheme w.o. selection of consensus protocol	×	✓	✓
The proposed scheme w.o. selection of block producer	✓	×	✓
The proposed scheme w.o. allocation of network bandwidth	✓	✓	×
Traditional scheme	×	×	×

Similarly, we assume the state of available bandwidth resources can be very good, good, medium, and bad. We set the transition probability matrix as

$$\Upsilon = \begin{bmatrix} 0.45 & 0.32 & 0.16 & 0.07 \\ 0.32 & 0.45 & 0.16 & 0.07 \\ 0.16 & 0.32 & 0.45 & 0.07 \\ 0.16 & 0.32 & 0.07 & 0.45 \end{bmatrix}. \quad (41)$$

The trust feature of the IoT system can be secure, medium, and compromised, whose transition probability matrix is

$$\mathcal{Z} = \begin{bmatrix} 0.5 & 0.3 & 0.2 \\ 0.3 & 0.5 & 0.2 \\ 0.2 & 0.3 & 0.5 \end{bmatrix}. \quad (42)$$

For comparison, there are five schemes simulated:

- 1) Proposed dueling QDL-based scheme with the selection of consensus protocol, the selection of block producer, and the allocation of bandwidth resources. We call it the proposed scheme.
- 2) Proposed scheme with the selection of block producer, and the allocation of bandwidth resources, but only using PBFT consensus protocol. We call it the proposed scheme w/o dynamic selection of consensus protocol.
- 3) Proposed scheme with the selection of consensus protocol, and the allocation of bandwidth resources, but randomly selecting a block producer. We call it the proposed scheme w/o dynamic selection of block producer.
- 4) Proposed scheme with the selection of consensus protocol, and the selection of block producer, but only using the local bandwidth resources. We call it the proposed scheme w/o dynamic allocation of network bandwidth.
- 5) Traditional scheme, which only uses PBFT consensus protocol, randomly selects a block producer, and only uses the local bandwidth resources.

The differences of these comparison schemes are presented in Table VIII.

In addition, our proposed service-oriented blockchain does not need any additional modifications regarding the whole blockchain. Only the transparent interface of consensus protocols is needed for the permissioned blockchain. As

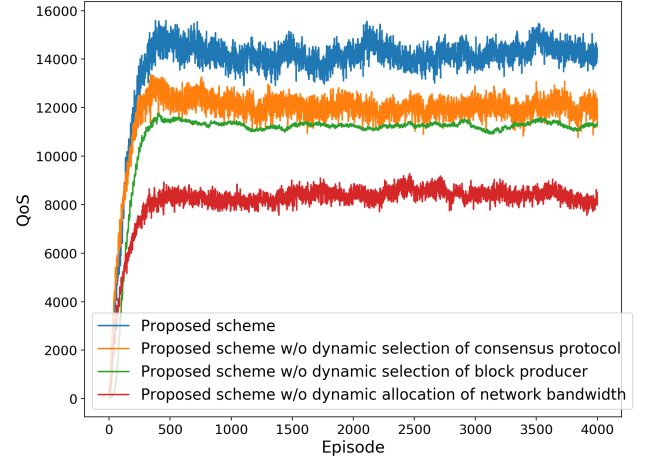


Fig. 8: The convergence performances under different schemes.

shown in [4], the author proposed a blockchain system with virtualization and decoupled control. Here, decoupling control from execution in blockchain was presented. The configuration of the blockchain system can be achieved by the decoupled control layer, such as the prioritization of transactions, the execution of smart contracts, and the selection of consensus protocols. By this system, the flexible deployment of consensus protocols is realizable.

B. Simulation Results

Fig. 8 shows the convergence performances under different schemes. At the beginning of the learning process, the QoS of each scheme is very low. With the increase of episodes, the QoS increases until a relatively stable value. It shows the convergence performance of our proposed scheme. From this figure, we can see that the QoS of each scheme is always less when the selection of consensus protocol, the selection of block producer, and allocating bandwidth resources are not jointly considered. This also shows the effectiveness of our proposed scheme.

Fig. 9 shows the convergence performance under different transition matrices with the proposed scheme. As we can see, the curves all converge to relatively stable values. The figure shows the convergence performance of our proposed scheme under different transition matrices. Specifically, we make four simulations under four transition probability matrices of user's latency preference, which are:

$$\mathcal{F}_2 = \begin{bmatrix} 0.45 & 0.35 & 0.2 \\ 0.35 & 0.45 & 0.2 \\ 0.2 & 0.35 & 0.45 \end{bmatrix}. \quad (43)$$

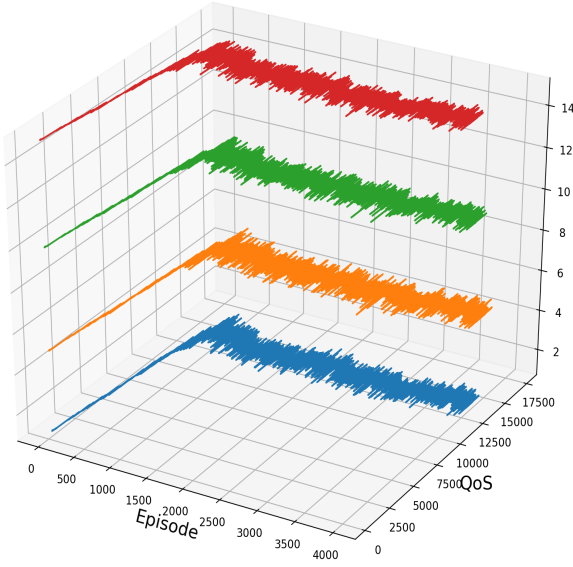


Fig. 9: The convergence performances under different transition matrices.

$$\mathcal{F}_2 = \begin{bmatrix} 0.75 & 0.22 & 0.03 \\ 0.22 & 0.75 & 0.03 \\ 0.03 & 0.22 & 0.75 \end{bmatrix}. \quad (44)$$

$$\mathcal{F}_2 = \begin{bmatrix} 0.5 & 0.32 & 0.18 \\ 0.32 & 0.5 & 0.18 \\ 0.18 & 0.32 & 0.5 \end{bmatrix}. \quad (45)$$

$$\mathcal{F}_2 = \begin{bmatrix} 0.65 & 0.25 & 0.1 \\ 0.25 & 0.65 & 0.1 \\ 0.1 & 0.25 & 0.65 \end{bmatrix}. \quad (46)$$

Fig. 10 shows the relationship between the QoS and the batch size in a block b . The bigger b enables to contain more transactions in one verification, which leads to the increase of the system throughput. However, more transactions in a block need more time to deliver, which reduces the system latency. Thus, the curves are lightly scaled. Under each batch size, our proposed scheme has the best QoS.

Fig. 11 presents the relationship between the QoS and the computation costs of verifying one signature θ . The bigger θ leads to more computation costs to verify one signature. Thus, the system throughput decreases. However, our proposed scheme still outperforms others.

VII. CONCLUSIONS AND FUTURE WORK

In this paper, we proposed a service-oriented permissioned blockchain in the IoT system. Rather than using a single consensus protocol as the best fit one, the proposed blockchain

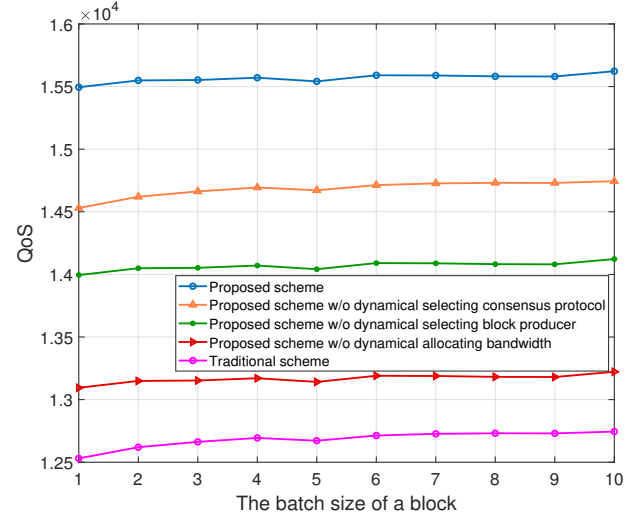


Fig. 10: The QoS versus the batch size in a block.

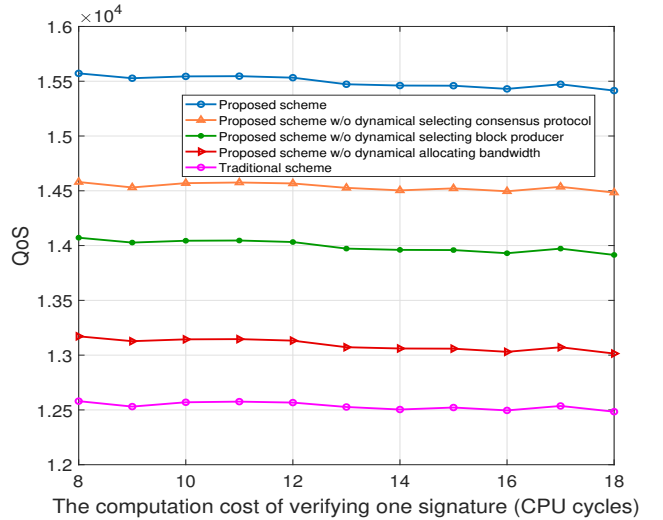


Fig. 11: The QoS versus the computation cost of verifying one signature.

used different consensus protocols to accommodate different users' QoS requirements. We quantified the performance of three representative consensus protocols, namely PBFT, Zyzzyva, and Quorum. Moreover, in order to improve the performance of the blockchain, the proposed blockchain can dynamically select block producers, and allocate network bandwidth to the blockchain. We formulated this joint optimization problem as a Markov decision process. Then we used a dueling deep reinforcement learning approach to solve this problem. Simulation results were presented to show the convergence and the effectiveness of our proposed scheme. The number of blockchain nodes is also an important aspect to improve the performance. Some future works

are in progress to solve this problem.

ACKNOWLEDGMENTS

We thank all reviewers for their detailed reviews and constructive suggestions, which have helped to greatly improve the quality of this article. This work was supported by National Engineering Laboratory for Public Safety Risk Perception and Control by Big Data (PSRPC).

REFERENCES

- [1] L. Chen, N. Zhao, Y. Chen, F. R. Yu, and G. Wei, "Over-the-air computation for IoT networks: Computing multiple functions with antenna arrays," *IEEE Internet of Things Journal*, pp. 1–1, 2018.
- [2] S. Gartner, "Forecast: The internet of things, worldwide, 2013," 2013.
- [3] O. Novo, "Blockchain meets IoT: An architecture for scalable access management in IoT," *IEEE Internet of Things Journal*, vol. 5, no. 2, pp. 1184–1195, April 2018.
- [4] F. R. Yu, "vDLT: A service-oriented blockchain system with virtualization and decoupled management/control and execution," *arXiv preprint arXiv:1809.00290*, 2018.
- [5] H. Yao, T. Mai, J. Wang, Z. Ji, C. Jiang, and Y. Qian, "Resource trading in blockchain-based industrial internet of things," *IEEE Transactions on Industrial Informatics*, 2019.
- [6] X. Li, D. Li, J. Wan, C. Liu, and M. Imran, "Adaptive transmission optimization in SDN-based industrial internet of things with edge computing," *IEEE Internet of Things Journal*, vol. 5, no. 3, pp. 1351–1360, June 2018.
- [7] "EOSIO," <https://eos.io>, Last Accessed: November 13, 2018.
- [8] "Zilliqa," <https://zilliqa.com/>, Last Accessed: November 15, 2018.
- [9] "Plasma," <http://plasma.io/>, Last Accessed: October 28, 2018.
- [10] P. Corcoran and S. K. Datta, "Mobile-edge computing and the internet of things for consumers: Extending cloud computing and services to the edge of the network," *IEEE Consumer Electronics Mag.*, vol. 5, no. 4, pp. 73–74, Oct 2016.
- [11] F. R. Yu, J. Liu, Y. He, P. Si, and Y. Zhang, "Virtualization for distributed ledger technology (vDLT)," *IEEE Access*, vol. 6, pp. 25 019–25 028, 2018.
- [12] M. Castro and B. Liskov, "Practical byzantine fault tolerance and proactive recovery," *ACM Trans. on Comp. Sys.*, vol. 20, no. 4, pp. 398–461, 2002.
- [13] R. Kotla, L. Alvisi, and M. D. et al., "Zyzyva: speculative byzantine fault tolerance[j]," *ACM SIGOPS Operating Sys. Review*, vol. 41, no. 6, pp. 45–58, 2007.
- [14] R. Guerraoui and V. Quema, "The next 700 BFT protocols[c]," 2010, pp. 363–376.
- [15] C. Qiu, H. Yao, C. Jiang, S. Guo, and F. Xu, "Cloud computing assisted blockchain-enabled internet of things," *IEEE Trans. on Cloud Comp.*, pp. 1–1, 2019.
- [16] X. Hao, L. Yu, L. Zhiqiang, L. Zhen, and G. Dawu, "Dynamic practical byzantine fault tolerance," in *Proc. Conf. on Commu. and Net. Security*, May 2018, pp. 1–8.
- [17] J. Liu, W. Li, G. Karame, and N. Asokan, "Scalable byzantine consensus via hardware-assisted secret sharing," *IEEE Trans. on Comp.*, pp. 1–1, 2018.
- [18] C. Qiu, F. R. Yu, H. Yao, C. Jiang, F. Xu, and C. Zhao, "Blockchain-based software-defined industrial internet of things: A dueling deep q-learning approach," *IEEE Internet of Things Journal*, 2018.
- [19] S. Rsch, I. Messadi, and R. Kapitza, "Towards low-latency byzantine agreement protocols using RDMA," in *Proc. Conf. on Dependable Sys. and Net. Workshops*, June 2018, pp. 146–151.
- [20] B. J. P. G. R. and S. A., "Making BFT protocols really adaptive[c]," in *Proc. Conf. on Parallel and Distributed*, 2015, pp. 904–913.
- [21] Y. Liu, A. H. Ngu, and L. Z. Zeng., "Qos computation and policing in dynamic web service selection," in *Proc. Conf. on Alternate Track Papers & Posters*, 2004, pp. 66–73.
- [22] C. Qiu, F. R. Yu, F. Xu, H. Yao, and C. Zhao, "Blockchain-based distributed software-defined vehicular networks via deep q-learning," in *Proc. Conf. on Design and Analysis of Intelligent Veh. Net. and App.*, 2018, pp. 8–14.
- [23] U. Klarman, S. Basu, A. Kuzmanovic, and E. G. Sirer, "bloxroute: A scalable trustless blockchain distribution network whitepaper."
- [24] H. Yao, T. Mai, X. Xu, P. Zhang, M. Li, and Y. Liu, "Network AI: An intelligent network architecture for self-learning control strategies in software defined networks," *IEEE Internet of Things Journal*, vol. 5, no. 6, pp. 4319–4327, Dec. 2018.
- [25] Z. Wang, T. Schaul, M. Hessel, H. Van Hasselt, M. Lanctot, and N. De Freitas, "Dueling network architectures for deep reinforcement learning," *arXiv preprint arXiv:1511.06581*, 2015.